Vocabulaire :

- Les arbres et le vocabulaire associé / arbre binaire / compression / compression de Huffman.

Prérequis :

- Avoir fait les séances précédentes sur les arbres et maîtrisé le vocabulaire.

Objectifs :

- Découvrir une application importante des arbres, la compression.
- Comprendre l'algorithme d'Huffman.

Cette séance est la dernière concernant la structure de données **arbres**.Tu as abbatu un sacré travail, bravo !

Pour finir, nous allons travaillé sur une application importante des arbres binaires : la compression de données sans perte, notamment la **compression de Huffman**.

I- POURQUOI COMPRESSER LES DONNÉES ?

De nos jours, le volume gigantesque de données échangées sur internet, et stockées sur des serveurs ne serait possible sans la compression des données.
En effet, une simple chanson pourrait prendre jusqu'à 150 Mo, ou 5 min de film 1Go, sans parler de l'ensemble des métadonnées, ...
Outre le gain d'espace, la compression améliore la vitesse de transmission des données sur les réseaux. Elle permet donc de réduire les coûts matériels de stockage, mais aussi les coûts liés à la bande passante réseau.
Elle donne lieu encore de nos jours à de nombreuses recherches en raison des enjeux économiques.

II - LES TYPES DE COMPRESSION

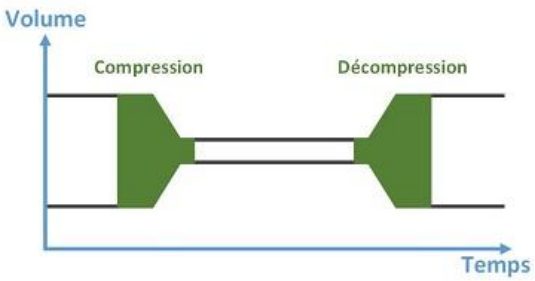
De nombreux algorithmes de compressions existent, chacun ayant sa particularité et surtout un type de données cible : images, vidéos, textes, exécutables, textes, ... En effet un algorithme de compression très efficace pour un fichier texte sera une catastrophe pour de la vidéo et inversement.

Lors de la décompression, la reconstruction des données s'effectue par un algorithme inverse ou différent à celui de la compression.

1- Compression symétrique / asymétrique

Symétrique : la compression et la décompression utilisent la même méthode, et nécessitent la même quantité de calculs.

Asymétrique : La compression et la décompression ne demandent pas la même quantité de travail. Par exemple la compression en MP3 est plus lente que la décompression pour la lecture, idem pour les vidéos.



2- Compression avec / sans Pertes

Sans Perte (lossless) :

Les algorithmes de compression sans pertes permettent une reconstitution exacte de l'information après le cycle de compression / décompression. Ils s'utilisent pour les fichiers contenant du texte, les programmes informatiques (.exe), …

S'il est facile de citer des formats de compression connus sans pertes (7z, Zip, Rar, Ace, Cab, gzip,...), on connaît moins le nom des algorithmes derrière ces extensions. Pour 7z par exemple, il s'agit du LZMA (pour Lempel-Ziv-Markov chain-Algorithm), pour zip du LZ77 et du codage Huffman, pour RAR du Lempel-Ziv et de la prédiction par reconnaissance partielle (PPM).

Avec perte (Lossy):

Lors de la compression, l'algorithme supprime irrémédiablement des données en essayant de conserver une qualité de lecture/visionnage optimale. On parle de compression irréversible.

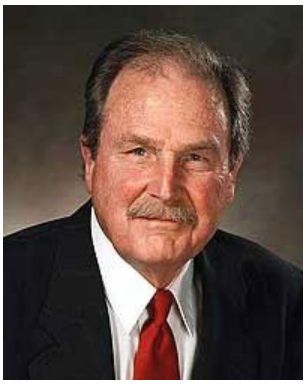
Les données dont la qualité se limite aux perceptions humaines (images, vidéos, sons, …) peuvent utiliser la compression avec pertes (JPEG, JPEG2000, fractales, MP3, MPEG...).

III- LES CRITÈRES DE COMPRESSION

La compression peut se définir par différents critères tels que :

- Le taux de compression (taille fichier compressé / taille fichier initial) qui s'exprime en pourcentage.Ou son inverse, le quotient (ratio) de compression = taille fichier initial / taille fichier compressé
- La vitesse de compression/décompression
- L'usage des ressources matérielles (mémoire, processeur)
- Rapport qualité/taux de compression (avec pertes)
- Robustesse : sensibilité de l'algorithme à des petites altérations du code compressé (erreurs de transmission)

IV- DAVID ALBERT HUFFMAN (1925 - 1999)



Le professeur Huffman a été un des pionniers dans le domaine de l'informatique.

Né dans l'Ohio, Huffman a obtenu une licence en ingénierie électrique à 18 ans. Puis il a travaillé pour la Marine militaire des États-Unis chargé de la maintenance des radars. Il a obtenu ensuite un master à l'université de l'Ohio et son doctorat au MIT en 1953.

Huffman contribua beaucoup au développement de la théorie de l'information et du codage, dont les découvertes sont à la base des systèmes de compression de fichiers informatiques dans toutes les machines de nos jours.

Il a également apporté sa contribution dans d'autres domaines tels que les signaux pour les radars par exemple.

Huffman a longtemps enseigné la théorie de l'information et l'analyse des signaux à l'université de Californie à Santa Cruz.

Afin de comprendre l'algorithme de compression d'Huffman, je vous invite pour cela à regarder la vidéo suivante : https://www.youtube.com/watch?v=Ml8br-DWqUQ

Malgré le son pour le moins pénible, elle permet d'abborder l'algorithme de façon concise et claire.

V- L'ALGORITHME DE COMPRESSION DE HUFFMAN

Le codage de Huffman est un codage statistique à longueur variable. David Albert Huffman l'a conçu alors qu'il était étudiant au MIT. Il publia l'algorithme en 1952 dans un article intitulé "A Method for the Construction of Minimum-Redundancy Codes".

La compression se fait en remplaçant les caractères les plus fréquents par des codes courts et les caractères les moins fréquents par des codes longs. On observe ainsi des réductions de taille de l'ordre de 20 à 90%. C'est une méthode de compression largement utilisée, souvent en complément d'autres algorithmes.

1- L'algorithme

Création d'un arbre binaire

1. À chaque caractère du fichier source, on associe sa fréquence (=nombre d'occurrence). Chaque caractère de fréquence non nulle représente un arbre binaire à un élément.
2. Classer chaque arbre binaire par valeur croissante ou décroissante de fréquence.
3. Enlever de la liste les 2 arbres binaires de plus faible fréquence. Rattacher ces 2 arbres à une nouvelle racine qui vaut la somme des fréquences des 2 arbres précédents.
4. Recommencer à 2, jusqu'à ce qu'il ne reste plus qu'un arbre binaire unique.

Codage des caractères

Pour connaître le code de Huffman associé à chaque caractère du fichier source, il suffit de parcourir l'arbre, en partant de la racine pour rejoindre le caractère voulu (feuille de l'arbre). À chaque nœud, on ajoute `0` au code en cours si on part dans le fils gauche et `1` si on part dans le fils droit.

Écriture du fichier compressé

On réécrit le fichier source en utilisant le nouveau code binaire des caractères, puis l'on joint la table de codage des caractères. En pratique c'est l'encodage de l'arbre de Huffman qui est joint avant le codage du texte compressé.

2- Utilisation

L'algorithme de compression de Huffman est utilisé par exemple, en complément d'autres algorithmes (souvent utilisé en deuxième passage), dans les formats de compression aussi variés que ZIP, PNG, JPEG, MP3.

3- Caractéristiques

- Algorithme simple
- Sans pertes
- Asymétrique
- Plus lent que d'autres algorithmes puisque compression en 2 temps (1ère lecture du fichier pour l'étude statistique, puis encodage et écriture du fichier compressé)

VI- RÉALISE LA COMPRESSION D'HUFFMAN SUR UN EXEMPLE

Le mot à compresser est « **BIENVENUE** ».

En t'appuyant sur le fichier `exemple_compression.pdf`, réponds aux différentes questions sur ce pdf.

1) Combien de bits sont utilisées en mémoire pour le mot « BIENVENUE » en codage ASCII ?

[Écris la réponse ici]

2) À quelle fréquence apparaisse chacune des lettres de ce mot ?

Symbole du fichier source						
Fréquence d'apparition						

3) Sur ton cahier, réalise l'arbre de Huffman.

4) Écris la table de codage optimale des caractères. Le code associé à chaque caractère du code source est le chemin d'accès de la racine à la feuille.

Symbole du fichier source						
Nouveau codage						

5) En utilisant la compression de Huffman, écris le mot « BIENVENUE » de manière codée.

[Écris la réponse ici]

6) Combien de bits sont utilisées en mémoire pour le mot « BIENVENUE » avec ce codage ?

[Écris la réponse ici]

7) Quel est le taux de compression quand on utilise l'algorithme de compression d'Huffman sur le mot bienvenue ?

[Écris la réponse ici]

En pratique on incorpore en entête du fichier compressé, le codage de l'arbre de Huffman afin de retrouver l'encodage optimal des symboles. Ce surplus est négligeable pour des fichiers de grandes tailles.

Nous avons à présent terminé nos séances sur les arbres, tu trouveras ci-dessous ton cadeau de Noël un peu en avance.

L'élève .....
a obtenu son diplome de meilleur(e) bucheron(ne) de France.
Ses compétences en arbres binaires et ABR ont impressionné le membre du jury.
Félicitations
pour le lycée Saint Exupéry,